300
SYSTEM DESIGN INTERVIEW QUESTIONS

Concise Answers for Every Concept
From Fundamentals to Real World System Design

buildwithbilal

Table of Contents

Chapter 1: Fundamentals of System Design (Questions 1 to 30)

Chapter 2: Networking and Protocols (Questions 31 to 55)

Chapter 3: Databases (Questions 56 to 91)

Chapter 4: Caching (Questions 92 to 111)

Chapter 5: Load Balancing and Scaling (Questions 112 to 136)

Chapter 6: Message Queues and Async Processing (Questions 137 to 156)

Chapter 7: API Design and Communication (Questions 157 to 176)

Chapter 8: Microservices and Architecture Patterns (Questions 177 to 201)

Chapter 9: Distributed Systems and Consistency (Questions 202 to 231)

Chapter 10: Security and Authentication (Questions 232 to 246)

Chapter 11: Monitoring, Logging, and Observability (Questions 247 to 261)

Chapter 12: Design Patterns and Best Practices (Questions 262 to 281)

Chapter 13: Real World System Design Problems (Questions 282 to 306)

Chapter 1: Fundamentals of System Design

Q1. What is system design?

System design is the process of defining the architecture, components, interfaces, and data flow of a system to meet specific requirements. It focuses on how different parts of a system work together to handle scale, reliability, and performance.

Q2. What is the difference between high level design and low level design?

High level design (HLD) defines the overall system architecture, major components, and how they interact. Low level design (LLD) zooms into individual components, covering class structures, method signatures, database schemas, and implementation details.

Q3. What are functional requirements?

Functional requirements describe what the system should do. For example, in a URL shortener: the system should accept a long URL and return a short one, and redirecting the short URL should take the user to the original.

Q4. What are non functional requirements?

Non functional requirements describe how the system should perform. These include latency (how fast), availability (how reliable), scalability (how much load), durability (data safety), and consistency (data accuracy).

Q5. What is latency?

Latency is the time taken for a request to travel from the client to the server and back with a response. It is usually measured in milliseconds. Lower latency means a faster user experience.

Q6. What is throughput?

Throughput is the number of requests or operations a system can handle per unit of time. It is typically measured in requests per second (RPS) or transactions per second (TPS).

Q7. What is availability?

Availability is the percentage of time a system is operational and accessible. It is calculated as uptime divided by total time. A system with 99.99% availability (four nines) allows roughly 52 minutes of downtime per year.

Q8. What is reliability?

Reliability is the probability that a system will perform its intended function without failure over a given period. A reliable system handles failures gracefully and does not lose data.

Q9. What is scalability?

Scalability is the ability of a system to handle increased load by adding resources. Vertical scaling adds more power to one machine. Horizontal scaling adds more machines.

Q10. What is vertical scaling?

Vertical scaling means upgrading the existing server with more CPU, RAM, or storage. It is simple but has an upper limit since hardware cannot grow indefinitely. It also creates a single point of failure.

Q11. What is horizontal scaling?

Horizontal scaling means adding more servers and distributing the load across them. It has no hard ceiling and improves fault tolerance, but requires load balancers, data replication, and stateless application design.

Q12. What is a single point of failure?

A single point of failure (SPOF) is any component whose failure would bring down the entire system. Good system design eliminates SPOFs by adding redundancy at every critical layer: servers, databases, network paths, and data centers.

Q13. What is redundancy?

Redundancy means duplicating critical components so that if one fails, another takes over. Examples include running multiple app servers behind a load balancer, keeping database replicas, and deploying across multiple data centers.

Q14. What is fault tolerance?

Fault tolerance is the ability of a system to continue operating when one or more components fail. It is achieved through redundancy, replication, health checks, and automatic failover mechanisms.

Q15. What is a distributed system?

A distributed system is a collection of independent computers that appear as a single system to the user. They communicate over a network and coordinate to achieve a common goal. Examples include Google Search, Netflix, and Amazon.

Q16. What are the challenges of distributed systems?

The main challenges are network failures (messages get lost or delayed), data consistency (keeping all copies in sync), partial failures (some nodes crash while others run), and coordination (agreeing on shared state across nodes).

Q17. What is back of the envelope estimation?

It is a rough calculation technique used in interviews to estimate system requirements like storage, bandwidth, and server count. For example: if 1 million users upload one 2MB photo daily, you need about 2TB of new storage per day.

Q18. What is the difference between stateful and stateless systems?

A stateless system does not store any client session data between requests. Each request contains all the information needed. A stateful system remembers past interactions. Stateless systems are easier to scale because any server can handle any request.

Q19. What is a monolithic architecture?

A monolith is a single unified codebase where all features are tightly coupled and deployed together. It is simple to develop and deploy initially but becomes hard to scale, maintain, and update as the application grows.

Q20. What is a microservices architecture?

Microservices break an application into small independent services, each responsible for one function. Each service can be developed, deployed, and scaled independently. The tradeoff is added complexity in communication, monitoring, and data management.

Q21. When should you choose a monolith over microservices?

Choose a monolith when you have a small team, the product is in early stages, speed of development matters more than scalability, and the domain is not well understood yet. Many successful companies started with a monolith and migrated later.

Q22. What is a proxy server?

A proxy server acts as an intermediary between a client and a server. A forward proxy hides the client's identity (like a VPN). A reverse proxy hides the server's identity and can handle load balancing, caching, and SSL termination.

Q23. What is a reverse proxy?

A reverse proxy sits in front of backend servers and forwards client requests to them. The client never communicates directly with the backend. Nginx and HAProxy are popular reverse proxies that also handle caching, compression, and security.

Q24. What does it mean to design for failure?

It means assuming that every component can and will fail, and building systems that detect, isolate, and recover from failures automatically. This includes using health checks, retries, circuit breakers, redundancy, and graceful degradation.

Q25. What is graceful degradation?

Graceful degradation means the system continues to function at a reduced capacity when some components fail, instead of crashing entirely. For example, if the recommendation engine goes down, the site still shows products without personalized suggestions.

Q26. What is the difference between concurrency and parallelism?

Concurrency means multiple tasks make progress by sharing time on the same resources (like threads on one CPU). Parallelism means multiple tasks run simultaneously on multiple resources (like threads on multiple CPU cores).

Q27. What is idempotency?

An operation is idempotent if performing it multiple times produces the same result as performing it once. GET, PUT, and DELETE are naturally idempotent. POST is not. Idempotency keys are used in payment systems to prevent duplicate charges.

Q28. What is a service level agreement (SLA)?

An SLA is a contract between a service provider and a customer that defines the expected level of service: uptime percentage, latency thresholds, and penalties for failing to meet them. For example, AWS guarantees 99.99% uptime for certain services.

Q29. What is the difference between SLA, SLO, and SLI?

SLI (Service Level Indicator) is the actual measured metric, like response time. SLO (Service Level Objective) is the target value for that metric, like p99 latency under 200ms. SLA (Service Level Agreement) is the formal contract built around SLOs.

Q30. What is capacity planning?

Capacity planning is the process of estimating the resources (servers, storage, bandwidth) needed to handle expected traffic. It involves analyzing growth trends, peak load patterns, and building headroom for unexpected traffic spikes.

Chapter 2: Networking and Protocols

Q31. What happens when you type a URL in the browser?

The browser performs a DNS lookup to find the IP address, establishes a TCP connection, performs a TLS handshake for HTTPS, sends an HTTP request, receives the response, and renders the HTML, CSS, and JavaScript on screen.

Q32. What is DNS?

DNS (Domain Name System) translates human readable domain names like google.com into IP addresses that computers use to identify each other. It works through a hierarchy of servers: browser cache, OS cache, ISP resolver, root server, TLD server, and authoritative server.

Q33. What is TCP?

TCP (Transmission Control Protocol) is a reliable, connection oriented protocol. It guarantees that data arrives in order and without errors by using acknowledgments and retransmissions. It is used for web traffic, email, and file transfers.

Q34. What is UDP?

UDP (User Datagram Protocol) is a faster but unreliable protocol. It does not guarantee delivery, ordering, or error checking. It is used where speed matters more than reliability: video streaming, online gaming, DNS lookups, and VoIP calls.

Q35. What is the difference between TCP and UDP?

TCP is reliable, ordered, and connection based. UDP is fast, unordered, and connectionless. Use TCP when you need guaranteed delivery (web pages, APIs). Use UDP when speed matters and losing some packets is acceptable (live video, gaming).

Q36. What is HTTP?

HTTP (Hypertext Transfer Protocol) is the protocol used for communication between web browsers and servers. It follows a request response model where the client sends a request (GET, POST, PUT, DELETE) and the server returns a response with a status code and data.

Q37. What is the difference between HTTP and HTTPS?

HTTP sends data as plain text. HTTPS encrypts data using TLS/SSL so it cannot be read by anyone intercepting the traffic. HTTPS protects passwords, payment information, and personal data from being stolen in transit.

Q38. What are HTTP status codes?

Status codes indicate the result of an HTTP request. 2xx means success (200 OK, 201 Created). 3xx means redirection (301 Moved, 304 Not Modified). 4xx means client error (400 Bad Request, 404 Not Found). 5xx means server error (500 Internal Error, 503 Unavailable).

Q39. What is REST?

REST (Representational State Transfer) is an architectural style for designing APIs over HTTP. It uses standard methods (GET, POST, PUT, DELETE), stateless communication, and resource based URLs like /users/123. Each request contains all the information the server needs.

Q40. What is the difference between REST and GraphQL?

REST uses fixed endpoints that return predefined data structures. GraphQL uses a single endpoint where the client specifies exactly what data it needs. GraphQL reduces over fetching and under fetching but adds complexity to the server.

Q41. What are WebSockets?

WebSockets provide a persistent, full duplex connection between client and server. Unlike HTTP where the client must initiate every request, WebSockets allow both sides to send data at any time. Used for chat apps, live notifications, and real time dashboards.

Q42. What is long polling?

Long polling is a technique where the client sends a request and the server holds the connection open until new data is available. Once data arrives, the server responds and the client immediately sends a new request. It simulates real time updates using HTTP.

Q43. What is the difference between WebSockets, long polling, and Server Sent Events?

WebSockets allow two way real time communication. Long polling simulates real time by holding HTTP connections open. SSE (Server Sent Events) allow the server to push updates to the client over a one way HTTP connection. WebSockets are best for bidirectional needs.

Q44. What is gRPC?

gRPC is a high performance RPC (Remote Procedure Call) framework built by Google. It uses Protocol Buffers for serialization (faster and smaller than JSON) and HTTP/2 for transport. It is commonly used for communication between microservices.

Q45. What is the difference between REST and gRPC?

REST uses JSON over HTTP/1.1 and is human readable. gRPC uses Protocol Buffers over HTTP/2, which is binary, faster, and supports streaming. REST is better for public APIs. gRPC is better for internal service to service communication where performance matters.

Q46. What is an IP address?

An IP address is a unique numerical label assigned to every device on a network. IPv4 uses 32 bit addresses (like 192.168.1.1) and IPv6 uses 128 bit addresses. Servers use IP addresses to identify where to send data.

Q47. What is a port?

A port is a number that identifies a specific process or service on a server. HTTP uses port 80, HTTPS uses 443, MySQL uses 3306. A single server can run multiple services by assigning each one a different port.

Q48. What is TLS/SSL?

TLS (Transport Layer Security) is the protocol that encrypts data between a client and a server. SSL is the older version, now deprecated. TLS involves a handshake where both sides agree on encryption keys before any data is exchanged.

Q49. What is a CDN?

A CDN (Content Delivery Network) is a network of servers distributed across multiple locations that cache and serve static content (images, videos, CSS, JS) from the server closest to the user. It reduces latency and offloads traffic from the origin server.

Q50. How does a CDN work?

When a user requests content, the CDN routes them to the nearest edge server. If that server has the content cached, it serves it directly. If not, it fetches from the origin server, caches it, and serves it. Popular CDNs include Cloudflare, AWS CloudFront, and Akamai.

Q51. What is a firewall?

A firewall monitors and controls incoming and outgoing network traffic based on predefined security rules. It acts as a barrier between trusted internal networks and untrusted external networks, blocking unauthorized access while allowing legitimate traffic.

Q52. What is bandwidth?

Bandwidth is the maximum amount of data that can be transmitted over a network connection per unit of time, measured in bits per second (Mbps, Gbps). Higher bandwidth means more data can flow simultaneously, but it does not guarantee low latency.

Q53. What is the OSI model?

The OSI model is a 7 layer conceptual framework for understanding network communication. From bottom to top: Physical, Data Link, Network, Transport, Session, Presentation, Application. Each layer handles a specific aspect of data transmission.

Q54. What is a VPN?

A VPN (Virtual Private Network) creates an encrypted tunnel between your device and a remote server. All your internet traffic passes through this tunnel, hiding your IP address and encrypting your data. It is essentially a forward proxy with encryption.

Q55. What is network latency and how do you reduce it?

Network latency is the delay between sending a request and receiving a response. Reduce it by using CDNs to serve content from nearby servers, caching frequently accessed data, minimizing the number of network hops, using efficient protocols, and compressing payloads.

Chapter 3: Databases

Q56. What is a relational database?

A relational database stores data in structured tables with rows and columns. Tables are linked through relationships using foreign keys. It uses SQL for querying and enforces ACID properties for reliable transactions. Examples: PostgreSQL, MySQL, Oracle.

Q57. What is a NoSQL database?

NoSQL databases store data in formats other than tables: documents (MongoDB), key value pairs (Redis), wide columns (Cassandra), or graphs (Neo4j). They offer flexible schemas, horizontal scaling, and high throughput at the cost of weaker consistency guarantees.

Q58. When should you use SQL vs NoSQL?

Use SQL when data is relational, consistency is critical, and you need complex queries with joins. Use NoSQL when you need flexible schemas, massive write throughput, horizontal scaling, or when data does not fit neatly into tables.

Q59. What are ACID properties?

Atomicity: a transaction either completes fully or not at all. Consistency: data always follows defined rules after a transaction. Isolation: concurrent transactions do not interfere with each other. Durability: once a transaction is committed, it survives crashes.

Q60. What is BASE?

BASE stands for Basically Available, Soft State, Eventually Consistent. It is the opposite philosophy to ACID. NoSQL databases often follow BASE, prioritizing availability and performance over strict consistency. Data may be temporarily inconsistent but will eventually converge.

Q61. What is database normalization?

Normalization is the process of organizing data to reduce redundancy. It splits data into related tables so each piece of information is stored in only one place. This prevents update anomalies but can make reads slower due to the need for joins.

Q62. What is denormalization?

Denormalization intentionally adds redundancy to a database by combining tables or duplicating data. It makes reads faster by avoiding joins but makes writes slower and more complex. It is commonly used in read heavy systems and data warehouses.

Q63. What is database indexing?

An index is a data structure (usually a B-Tree or hash) that speeds up lookups on a specific column. Without an index, the database scans every row. With an index, it jumps directly to the matching rows. The tradeoff is slower writes and extra storage.

Q64. What is a B-Tree index?

A B-Tree is a self balancing tree structure used by most relational databases for indexing. It keeps data sorted and allows searches, insertions, and deletions in logarithmic time. It works well for range queries and equality lookups.

Q65. What is a composite index?

A composite index is an index on multiple columns. The order of columns matters. An index on (city, age) is useful for queries filtering by city, or by city and age together, but not for queries filtering by age alone.

Q66. What is database sharding?

Sharding splits a large database into smaller pieces (shards), each stored on a different server. Each shard holds a subset of the data. This distributes the load and allows the system to scale horizontally. The challenge is choosing the right shard key.

Q67. What are common sharding strategies?

Range based sharding assigns rows by value ranges (users A through M on shard 1). Hash based sharding applies a hash function to distribute rows evenly. Geographic sharding places data on servers closest to users. Each strategy has different tradeoffs for hotspots and cross shard queries.

Q68. What is database replication?

Replication creates copies of a database on multiple servers. The primary handles writes, and replicas handle reads. This improves read performance and provides fault tolerance. If the primary fails, a replica can be promoted.

Q69. What is master slave replication?

One master database handles all writes. One or more slave databases replicate the master's data and handle reads. This scales read traffic but the master is still a single point of failure for writes.

Q70. What is master master replication?

Both databases can handle reads and writes. Data syncs between them. This provides write redundancy but introduces complexity: write conflicts can occur when both masters update the same data simultaneously.

Q71. What is replication lag?

Replication lag is the delay between a write on the primary database and that write appearing on the replica. During this window, reading from the replica returns stale data. Critical systems like banking require techniques to handle this.

Q72. What is a write ahead log (WAL)?

A WAL records every database change to a log file before applying it to the actual data. If the system crashes mid operation, it replays the log on restart to recover. PostgreSQL, MySQL, and Kafka all use WAL for durability and replication.

Q73. What is a transaction?

A database transaction is a sequence of operations that must all succeed or all fail together. If any step fails, all changes are rolled back. This prevents partial updates that would leave data in an inconsistent state.

Q74. What is optimistic locking?

Optimistic locking allows multiple transactions to proceed without locking resources. Before committing, it checks if the data was modified by another transaction (using a version number). If yes, the transaction fails and must be retried.

Q75. What is pessimistic locking?

Pessimistic locking locks the data row when a transaction reads it, preventing other transactions from modifying it until the lock is released. It is safer under high contention but reduces concurrency and can cause deadlocks.

Q76. What is a deadlock?

A deadlock occurs when two transactions each hold a lock that the other needs. Neither can proceed, and both wait forever. Databases detect deadlocks and kill one transaction to resolve the situation.

Q77. What is connection pooling?

Connection pooling reuses a set of pre established database connections instead of creating a new connection for every request. Creating connections is expensive. Pooling improves performance and prevents the database from being overwhelmed by too many simultaneous connections.

Q78. What is a stored procedure?

A stored procedure is a precompiled set of SQL statements stored in the database. It runs inside the database engine, reducing network round trips. However, it ties business logic to the database and makes code harder to test and maintain.

Q79. What is an ORM?

An ORM (Object Relational Mapping) lets you interact with a database using your programming language's objects instead of writing raw SQL. Examples: Hibernate (Java), SQLAlchemy (Python), Prisma (Node.js). It is convenient but can generate inefficient queries at scale.

Q80. What is a document database?

A document database stores data as flexible JSON like documents. Each document can have a different structure. It is ideal for content management, user profiles, and catalogs where the schema varies. MongoDB and CouchDB are popular examples.

Q81. What is a key value store?

A key value store maps unique keys to values. It is the simplest NoSQL type, optimized for fast reads and writes. Use cases include caching (Redis), session storage, and configuration data. It does not support complex queries or relationships.

Q82. What is a wide column database?

A wide column database stores data in rows and dynamic columns. Each row can have a different set of columns. Cassandra and HBase are examples. They excel at handling massive write loads and time series data across distributed clusters.

Q83. What is a graph database?

A graph database stores data as nodes (entities) and edges (relationships). It is optimized for traversing connections. Use cases include social networks, recommendation engines, and fraud detection. Neo4j and Amazon Neptune are popular choices.

Q84. What is a time series database?

A time series database is optimized for storing and querying timestamped data. Examples: InfluxDB, TimescaleDB. Use cases include monitoring system metrics, IoT sensor data, and financial market data where data is written continuously and queried by time range.

Q85. What is database partitioning?

Partitioning divides a large table into smaller pieces. Horizontal partitioning splits by rows (same as sharding). Vertical partitioning splits by columns, keeping frequently accessed columns together. Both improve query performance on large datasets.

Q86. What is a materialized view?

A materialized view is a precomputed query result stored as a physical table. Unlike a regular view that runs the query every time, a materialized view is already calculated. It speeds up complex read queries but needs to be refreshed when underlying data changes.

Q87. What is polyglot persistence?

Polyglot persistence means using different database technologies for different parts of the same application based on their strengths. For example: PostgreSQL for user accounts, Redis for caching, MongoDB for product reviews, and Elasticsearch for search.

Q88. What is an OLTP database?

OLTP (Online Transaction Processing) databases are optimized for fast, frequent, small read/write transactions. They power day to day operations like placing orders and updating accounts. Examples: PostgreSQL, MySQL.

Q89. What is an OLAP database?

OLAP (Online Analytical Processing) databases are optimized for complex analytical queries over large datasets. They are used for business intelligence, reporting, and data warehousing. Examples: Snowflake, BigQuery, Redshift.

Q90. What is eventual consistency?

Eventual consistency means that after a write, all replicas will eventually have the same data, but there may be a brief window where some replicas return stale data. It is acceptable for social media feeds but not for bank account balances.

Q91. What is strong consistency?

Strong consistency guarantees that every read returns the most recent write. All nodes agree on the current value before responding. It is slower because it requires coordination between nodes but is necessary for financial and transactional systems.

Chapter 4: Caching

Q92. What is caching?

Caching stores a copy of frequently accessed data in a faster storage layer (usually memory) so future requests can be served without hitting the slower source. It reduces latency, lowers database load, and improves throughput.

Q93. Where can caching be applied?

Caching can be applied at multiple levels: browser cache (client side), CDN cache (edge servers), application cache (Redis/Memcached), database query cache, and even CPU cache. Each level intercepts requests before they reach the slower layer behind it.

Q94. What is Redis?

Redis is an open source in memory data store used as a cache, message broker, and database. It stores data as key value pairs in RAM for extremely fast access. It supports data structures like strings, lists, sets, hashes, and sorted sets.

Q95. What is Memcached?

Memcached is a simple, high performance in memory caching system. It stores key value pairs and is designed for speed and simplicity. Unlike Redis, it does not support complex data structures, persistence, or replication natively.

Q96. What is the difference between Redis and Memcached?

Redis supports complex data types, persistence, replication, and pub/sub. Memcached is simpler, multithreaded, and slightly faster for basic key value lookups. Use Redis when you need features beyond simple caching. Use Memcached for pure, simple caching at scale.

Q97. What is cache invalidation?

Cache invalidation is the process of removing or updating stale data from the cache when the underlying source data changes. It is one of the hardest problems in computer science because serving stale data leads to bugs while invalidating too aggressively hurts performance.

Q98. What is TTL in caching?

TTL (Time To Live) is the duration for which a cached item is considered valid. After the TTL expires, the item is automatically removed. Short TTLs keep data fresh but increase cache misses. Long TTLs improve performance but risk serving stale data.

Q99. What is a cache aside pattern?

In cache aside (lazy loading), the application checks the cache first. On a miss, it reads from the database, stores the result in the cache, and returns it. Data is only cached when requested. Stale data is possible unless you invalidate on writes.

Q100. What is write through cache?

In write through caching, every write goes to both the cache and the database simultaneously. This keeps the cache always up to date but adds latency to writes since both stores must be updated before confirming success.

Q101. What is write behind (write back) cache?

In write behind caching, the application writes only to the cache. The cache asynchronously writes to the database in the background. This makes writes very fast but risks data loss if the cache crashes before the data is persisted.

Q102. What is a cache stampede?

A cache stampede (thundering herd) occurs when a popular cache entry expires and many requests simultaneously hit the database to reload it. Solutions include using locks so only one request refills the cache, staggered TTLs, or pre warming the cache before expiry.

Q103. What is a distributed cache?

A distributed cache spreads cached data across multiple servers. This allows caching more data than a single server's memory can hold and improves availability. Consistent hashing is often used to determine which server holds each key.

Q104. What is cache warming?

Cache warming is the process of preloading the cache with expected high traffic data before it is actually requested. This prevents a burst of cache misses when a system starts up or after a cache flush.

Q105. What is the LRU eviction policy?

LRU (Least Recently Used) removes the item that has not been accessed for the longest time when the cache is full. It assumes recently accessed data is likely to be accessed again. It is the most commonly used eviction policy.

Q106. What is the LFU eviction policy?

LFU (Least Frequently Used) removes the item that has been accessed the fewest number of times. It works well when some items are consistently popular but can be slow to adapt when access patterns change.

Q107. When should you NOT use caching?

Avoid caching when data changes very frequently and must always be fresh, when the dataset is too large to cache effectively, when cache hit rates are low (data is rarely re requested), or when the added complexity of cache invalidation is not justified.

Q108. What is a cache hit and a cache miss?

A cache hit occurs when the requested data is found in the cache and served directly. A cache miss occurs when the data is not in the cache and must be fetched from the source. The cache hit ratio measures caching effectiveness.

Q109. How do you handle cache failures?

Design the system so that a cache failure is not catastrophic. The application should fall back to the database on cache misses. Use multiple cache nodes for redundancy. Monitor cache health and have alerts for sudden drops in hit rate.

Q110. What is a read through cache?

In a read through cache, the cache itself is responsible for loading data from the database on a miss. The application only ever talks to the cache. This simplifies application code but couples the cache tightly to the database.

Q111. How do you decide what to cache?

Cache data that is read frequently but written rarely, is expensive to compute or fetch, does not need to be perfectly fresh, and has a high cache hit potential. Examples: user profiles, product catalogs, session tokens, and configuration data.

Chapter 5: Load Balancing and Scaling

Q112. What is a load balancer?

A load balancer distributes incoming network traffic across multiple servers so no single server is overwhelmed. It improves responsiveness, increases availability, and enables horizontal scaling. It sits between the client and the server pool.

Q113. What is round robin load balancing?

Round robin sends each new request to the next server in a circular order. Server 1, then 2, then 3, then back to 1. It is simple but does not account for differences in server capacity or current load.

Q114. What is least connections load balancing?

Least connections sends each new request to the server with the fewest active connections. It works better than round robin when requests have varying processing times because it naturally avoids overloading busy servers.

Q115. What is weighted round robin?

Weighted round robin assigns more traffic to more powerful servers. A server with weight 3 receives three times the traffic of a server with weight 1. This is useful when servers have different hardware specifications.

Q116. What is IP hash load balancing?

IP hash uses the client's IP address to determine which server receives the request. The same client always goes to the same server. This is useful for session affinity when sessions are stored locally on the server.

Q117. What is a Layer 4 load balancer?

A Layer 4 (transport layer) load balancer routes traffic based on IP address and port number without inspecting the content. It is faster because it makes routing decisions without reading the request body. Examples: AWS NLB, HAProxy in TCP mode.

Q118. What is a Layer 7 load balancer?

A Layer 7 (application layer) load balancer inspects the HTTP request content (URL, headers, cookies) to make routing decisions. It can route /api requests to one server pool and /static to another. Examples: Nginx, AWS ALB.

Q119. What is the difference between Layer 4 and Layer 7 load balancers?

Layer 4 is faster because it routes based on IP/port without reading content. Layer 7 is smarter because it can route based on URL path, headers, or cookies. Use Layer 4 for raw speed and Layer 7 for content based routing.

Q120. What is session affinity (sticky sessions)?

Session affinity ensures that all requests from the same user go to the same server. This is needed when session data is stored locally on the server. The downside is uneven load distribution and reduced fault tolerance.

Q121. What is health checking in load balancers?

Load balancers periodically send health check requests (pings or HTTP requests) to each server. If a server fails to respond, the load balancer stops sending traffic to it. When it recovers, traffic is gradually restored.

Q122. What is auto scaling?

Auto scaling automatically adds or removes servers based on traffic demand. When CPU usage or request count crosses a threshold, new servers spin up. When demand drops, excess servers are terminated. Cloud platforms like AWS, GCP, and Azure offer this natively.

Q123. What is the difference between scaling up and scaling out?

Scaling up (vertical) means adding more resources to one server. Scaling out (horizontal) means adding more servers. Scaling up is simpler but limited. Scaling out is more complex but has no hard ceiling and provides redundancy.

Q124. What is a global server load balancer (GSLB)?

A GSLB distributes traffic across data centers in different geographic regions. It uses DNS based routing to direct users to the nearest or healthiest data center. This improves latency for global users and provides disaster recovery across regions.

Q125. What is connection draining?

Connection draining (deregistration delay) allows a server being removed from the pool to finish processing its active requests before being taken offline. This prevents abruptly dropping user requests during deployments or scaling events.

Q126. How do load balancers handle SSL?

Load balancers can terminate SSL/TLS connections, decrypting HTTPS traffic at the load balancer and forwarding plain HTTP to backend servers. This offloads the encryption workload from application servers and simplifies certificate management.

Q127. What is rate limiting?

Rate limiting controls how many requests a user or client can make in a given time window. It protects servers from abuse, DDoS attacks, and runaway scripts. Common algorithms include token bucket, leaky bucket, fixed window, and sliding window.

Q128. What is the token bucket algorithm?

Each user gets a bucket that fills with tokens at a fixed rate. Each request consumes one token. If the bucket is empty, the request is rejected. The bucket has a maximum capacity that allows for short bursts. Used by AWS and Stripe.

Q129. What is the leaky bucket algorithm?

Requests enter a bucket and leak out at a constant rate. If the bucket overflows, new requests are dropped. This smooths out bursty traffic into a steady flow. It is simpler than token bucket but less flexible for allowing bursts.

Q130. What is the sliding window rate limiter?

A sliding window tracks requests in a rolling time period rather than fixed intervals. This avoids the spike problem at window boundaries that fixed window has. It provides smoother and more accurate rate limiting at the cost of slightly more memory.

Q131. How do you rate limit in a distributed system?

Use a centralized store like Redis to track request counts across all servers. Each server checks and increments the counter in Redis before processing a request. This ensures consistent rate limiting even when requests hit different servers.

Q132. What is throttling?

Throttling is the process of slowing down or limiting the rate of requests. Unlike rate limiting which rejects excess requests, throttling can queue them or respond with reduced quality. It protects backend systems during traffic spikes.

Q133. What is a content delivery network and how does it help with scaling?

A CDN caches static content on edge servers worldwide. By serving images, videos, and static files from servers close to users, it reduces latency and offloads up to 90% of traffic from your origin servers, dramatically reducing the scaling burden on your backend.

Q134. What is elastic scaling?

Elastic scaling is the ability to automatically scale resources up or down based on real time demand. It is a key feature of cloud computing, allowing you to pay only for what you use while handling traffic spikes without manual intervention.

Q135. What happens if a load balancer itself fails?

Production systems use redundant load balancers in an active passive or active active configuration. If the primary fails, the secondary takes over. Cloud load balancers (AWS ELB, GCP LB) are managed services with built in redundancy.

Q136. What is the difference between a load balancer and a reverse proxy?

A load balancer distributes traffic across multiple servers. A reverse proxy can do that plus handle caching, SSL termination, compression, and security. Every load balancer is a reverse proxy, but not every reverse proxy is a load balancer.

Chapter 6: Message Queues and Async Processing

Q137. What is a message queue?

A message queue is a middleware component where producers send messages and consumers pick them up for processing. The queue decouples the producer from the consumer, allowing them to work at different speeds and fail independently.

Q138. Why use message queues?

Message queues decouple services, smooth out traffic spikes, enable async processing (respond to users immediately while background tasks run), provide reliability (messages persist until processed), and allow independent scaling of producers and consumers.

Q139. What is Apache Kafka?

Kafka is a distributed event streaming platform. It stores messages in ordered, partitioned logs called topics. It is designed for high throughput, durability, and real time data streaming. Used by LinkedIn, Uber, and Netflix for processing millions of events per second.

Q140. What is the difference between Kafka and RabbitMQ?

Kafka is a log based system built for high throughput streaming and event replay. RabbitMQ is a traditional message broker with flexible routing, message acknowledgment, and priority queues. Use Kafka for event streaming. Use RabbitMQ for task queues and complex routing.

Q141. What is a topic in Kafka?

A topic is a named channel where messages are published. Topics are split into partitions for parallelism. Each partition is an ordered, append only log. Consumers read from partitions independently, enabling high throughput.

Q142. What is a consumer group in Kafka?

A consumer group is a set of consumers that cooperatively read from a topic. Each partition is assigned to exactly one consumer in the group. If a consumer fails, its partitions are reassigned to other consumers in the group automatically.

Q143. What is the producer consumer pattern?

A producer creates messages and puts them in a queue. A consumer picks messages from the queue and processes them. This pattern decouples the two sides: the producer does not wait for the consumer, and both can scale independently.

Q144. What is the pub sub pattern?

In publish subscribe, publishers send messages to a topic without knowing who will receive them. Subscribers register interest in specific topics and receive all messages published to those topics. One message is delivered to all subscribers.

Q145. What is the difference between a message queue and pub sub?

In a message queue, each message is consumed by exactly one consumer (point to point). In pub sub, each message is broadcast to all subscribers. Use queues for task distribution. Use pub sub when multiple services need to react to the same event.

Q146. What is dead letter queue?

A dead letter queue (DLQ) stores messages that could not be processed after a defined number of retries. Instead of losing failed messages or retrying forever, they are moved to the DLQ for investigation and manual reprocessing.

Q147. What is message ordering?

Message ordering guarantees that messages are processed in the order they were sent. Kafka maintains order within a partition but not across partitions. For strict ordering, all related messages must go to the same partition using the same key.

Q148. What is at least once delivery?

At least once delivery guarantees that every message is delivered at least one time, but it may be delivered more than once. The consumer must be idempotent to handle duplicates. This is the default mode in most message systems.

Q149. What is at most once delivery?

At most once delivery means a message is delivered zero or one time. It may be lost but never duplicated. This is the fastest option but suitable only when occasional message loss is acceptable, like logging non critical metrics.

Q150. What is exactly once delivery?

Exactly once delivery guarantees each message is processed exactly one time. It is the hardest to achieve and usually requires coordination between the queue and the consumer using transactions or idempotency keys. Kafka supports exactly once semantics within its ecosystem.

Q151. What is backpressure?

Backpressure is a mechanism where a slow consumer signals the producer to slow down or stop sending messages. Without backpressure, the queue grows unbounded and eventually runs out of memory. It prevents the system from being overwhelmed.

Q152. What is Amazon SQS?

Amazon SQS (Simple Queue Service) is a fully managed message queue service by AWS. It supports standard queues (best effort ordering, at least once delivery) and FIFO queues (strict ordering, exactly once processing). It scales automatically.

Q153. What is event sourcing?

Event sourcing stores every state change as an immutable event instead of storing just the current state. To get the current state, you replay all events. This provides a complete audit trail and allows rebuilding state from any point in time.

Q154. When should you use event sourcing?

Use event sourcing when you need a full audit trail (financial systems), when you need to rebuild state from history, or when multiple consumers need to react to state changes differently. Avoid it for simple CRUD applications where the added complexity is not justified.

Q155. What is CQRS?

CQRS (Command Query Responsibility Segregation) separates the read model from the write model. Commands (writes) go to one data store optimized for writes. Queries (reads) go to another store optimized for reads. Events keep them in sync.

Q156. How do message queues help with scaling?

Message queues absorb traffic spikes by buffering requests. Instead of your backend crashing under sudden load, messages wait in the queue and are processed at a sustainable rate. You can add more consumers to drain the queue faster during peaks.

Chapter 7: API Design and Communication

Q157. What is an API?

An API (Application Programming Interface) is a contract that defines how two software components communicate. It specifies what requests can be made, what data formats are used, and what responses to expect. APIs are the backbone of modern software architecture.

Q158. What are the key principles of good API design?

Good APIs are consistent (use standard conventions), predictable (similar resources behave similarly), versioned (changes do not break existing clients), well documented, idempotent where appropriate, and return meaningful error messages with proper HTTP status codes.

Q159. What is API versioning and why is it important?

API versioning allows you to make breaking changes without affecting existing clients. Common approaches: URL path versioning (/v1/users), query parameter versioning (?version=1), or header versioning (Accept: application/v1+json). URL path versioning is the most common.

Q160. What is pagination in APIs?

Pagination returns data in smaller chunks instead of sending millions of records at once. Common approaches: offset based (page=2&limit=20), cursor based (after=xyz123), and keyset based. Cursor based is best for real time data because it handles insertions and deletions gracefully.

Q161. What is API rate limiting?

API rate limiting restricts how many requests a client can make in a given time window. It protects servers from abuse and ensures fair usage. When exceeded, the server returns HTTP 429 (Too Many Requests) with a Retry-After header.

Q162. What is an API Gateway?

An API Gateway is a single entry point for all client requests in a microservices architecture. It handles routing, authentication, rate limiting, request transformation, response aggregation, and logging. Popular tools: Kong, AWS API Gateway, Nginx.

Q163. What is the difference between an API Gateway and a load balancer?

A load balancer distributes traffic across servers. An API Gateway does that plus handles authentication, rate limiting, request routing to different services, response aggregation, protocol translation, and logging. The gateway sits at the application layer with business awareness.

Q164. What is REST and what makes an API RESTful?

A RESTful API follows REST principles: uses standard HTTP methods (GET, POST, PUT, DELETE), is stateless (each request contains all needed information), uses resource based URLs (/users/123), and returns standard HTTP status codes.

Q165. What is GraphQL?

GraphQL is a query language for APIs where the client specifies exactly what data it needs. Unlike REST where each endpoint returns a fixed structure, GraphQL has one endpoint and the client defines the response shape. It solves over fetching and under fetching.

Q166. When should you use GraphQL vs REST?

Use REST for simple CRUD operations, public APIs, and when caching matters (REST works well with HTTP caching). Use GraphQL when clients need flexible queries, when you want to reduce the number of API calls, or when different clients need different data shapes.

Q167. What is gRPC and when should you use it?

gRPC is a high performance RPC framework using Protocol Buffers and HTTP/2. It supports streaming and is much faster than REST for service to service communication. Use it for internal microservice communication where performance and type safety matter.

Q168. What is idempotency in APIs?

An idempotent API produces the same result regardless of how many times the same request is made. GET, PUT, and DELETE are naturally idempotent. POST is not. For non idempotent operations like payments, use idempotency keys to prevent duplicate processing.

Q169. What is HATEOAS?

HATEOAS (Hypermedia As The Engine Of Application State) is a REST constraint where API responses include links to related actions. For example, a response for an order might include links to cancel, update, or track it. In practice, few APIs implement this fully.

Q170. What is an API contract?

An API contract is a formal specification of how the API behaves: endpoints, request/response formats, authentication, error codes, and rate limits. Tools like OpenAPI (Swagger) let you define contracts that can generate documentation and client code.

Q171. What is webhook?

A webhook is a callback mechanism where the server sends an HTTP POST request to a client specified URL when an event occurs. Instead of the client polling repeatedly for updates, the server pushes updates. Used by Stripe for payment notifications and GitHub for repo events.

Q172. What is the difference between synchronous and asynchronous communication?

Synchronous: the caller sends a request and waits for a response before proceeding. Asynchronous: the caller sends a message and continues without waiting. Use synchronous for real time needs. Use asynchronous (via queues) for background processing and decoupling.

Q173. What is request response vs event driven communication?

Request response is synchronous: client asks, server answers. Event driven is asynchronous: a service emits an event, and any interested service reacts to it. Event driven enables loose coupling and better scalability but adds complexity in tracing and debugging.

Q174. What is payload compression in APIs?

Payload compression reduces the size of API responses using algorithms like gzip or brotli. The client sends an Accept-Encoding header, and the server compresses the response. This reduces bandwidth usage and improves response time, especially on slow networks.

Q175. What is CORS?

CORS (Cross Origin Resource Sharing) is a browser security mechanism that controls which domains can access your API. By default, browsers block requests to different domains. The server must explicitly allow cross origin requests by setting the appropriate headers.

Q176. What is API throttling vs rate limiting?

Rate limiting rejects requests that exceed a threshold. Throttling slows them down by queuing or delaying. Rate limiting is binary (allow or reject). Throttling is gradual (process slower). Both protect backend systems from being overwhelmed.

Chapter 8: Microservices and Architecture Patterns

Q177. What are microservices?

Microservices are an architectural style where an application is composed of small, independent services. Each service handles one business function, has its own database, communicates via APIs, and can be deployed and scaled independently.

Q178. What are the advantages of microservices?

Independent deployment (update one service without redeploying everything), independent scaling (scale only the services under heavy load), technology flexibility (each service can use the best language/database for its needs), and fault isolation (one service crashing does not bring down the whole system).

Q179. What are the disadvantages of microservices?

Increased operational complexity, distributed system challenges (network failures, data consistency), harder debugging and testing, service discovery and communication overhead, and the need for robust monitoring, logging, and tracing infrastructure.

Q180. What is service discovery?

Service discovery is how services find each other in a dynamic environment where instances scale up and down. Services register themselves on startup and deregister on shutdown. Others look up the registry to find available instances. Tools: Consul, Eureka, Kubernetes DNS.

Q181. What is the difference between client side and server side service discovery?

In client side discovery, the calling service queries the registry directly and picks an instance. In server side discovery, the caller sends requests to a router or load balancer that queries the registry and forwards the request. Kubernetes uses server side discovery.

Q182. What is the circuit breaker pattern?

A circuit breaker monitors calls to an external service. If failures exceed a threshold, it trips open and immediately rejects further calls without attempting them. After a cooldown period, it lets a few test calls through. If they succeed, it closes again. This prevents cascading failures.

Q183. What is the saga pattern?

A saga is a sequence of local transactions across multiple microservices. Each step has a compensating transaction to undo its work if a later step fails. Instead of distributed ACID transactions, sagas achieve eventual consistency through coordinated local transactions.

Q184. What is the difference between orchestration and choreography in sagas?

In orchestration, a central coordinator tells each service what to do and when. In choreography, each service listens for events and decides independently what to do next. Orchestration is easier to understand. Choreography is more loosely coupled but harder to debug.

Q185. What is the sidecar pattern?

A sidecar is a helper container deployed alongside the main application container. It handles cross cutting concerns like logging, monitoring, service mesh communication, or configuration management without modifying the application code.

Q186. What is a service mesh?

A service mesh is an infrastructure layer that handles service to service communication. It manages routing, load balancing, security (mTLS), retries, and observability between microservices using sidecar proxies. Popular tools: Istio, Linkerd, Envoy.

Q187. What is the strangler fig pattern?

The strangler fig pattern is a migration strategy for moving from a monolith to microservices. You gradually replace specific pieces of the monolith with new microservices while routing traffic through a facade. Over time, the monolith shrinks and eventually disappears.

Q188. What is the bulkhead pattern?

The bulkhead pattern isolates different parts of a system so that a failure in one does not bring down others. Like compartments in a ship, each service or resource pool is isolated. If the email service overwhelms its thread pool, the payment service is unaffected.

Q189. What is the ambassador pattern?

An ambassador is a helper service that handles outgoing network requests on behalf of the main application. It manages retries, circuit breaking, logging, and monitoring. It offloads network resilience logic from the application code.

Q190. What is the API composition pattern?

API composition aggregates data from multiple microservices into a single response. A composer service calls multiple downstream services, combines their responses, and returns one unified result to the client. The API Gateway often serves this role.

Q191. What is the Database per Service pattern?

Each microservice owns and manages its own database. No other service can access that database directly. Services communicate through APIs. This ensures loose coupling but makes cross service queries and transactions harder.

Q192. What is the Shared Database antipattern?

Multiple microservices sharing the same database defeats the purpose of microservices. Changes to the database schema affect all services. It creates tight coupling and makes independent deployment impossible. Avoid this pattern in microservices architectures.

Q193. What is domain driven design (DDD)?

DDD is a software design approach that structures the codebase around business domains. It defines bounded contexts (clear boundaries between subsystems), entities (objects with identity), value objects, aggregates, and domain events. It helps define clean microservice boundaries.

Q194. What is a bounded context?

A bounded context is a clear boundary within which a particular domain model applies. In an e-commerce system, the order context and the inventory context may both have a Product entity, but each defines it differently based on its own needs.

Q195. What is the Backend for Frontend (BFF) pattern?

BFF creates a separate backend service for each type of client (mobile, web, desktop). Each BFF tailors the API to its client's specific needs instead of forcing all clients to use the same generic API. This reduces over fetching and simplifies client code.

Q196. How do microservices handle distributed transactions?

Microservices avoid traditional distributed transactions because they are slow and create tight coupling. Instead, they use sagas (compensating transactions), event driven communication, and eventual consistency to coordinate state changes across services.

Q197. What is a monolith first approach?

Start by building a monolith to understand your domain and get to market quickly. As the product grows and boundaries become clear, gradually extract microservices. This avoids premature complexity and lets you make informed decisions about service boundaries.

Q198. What is the retry pattern?

When a call to an external service fails, retry the request after a short delay. Use exponential backoff (double the wait time each retry) and add jitter (random delay) to prevent all clients from retrying at the same time and overwhelming the service.

Q199. What is the outbox pattern?

The outbox pattern ensures reliable event publishing. When a service updates its database, it also writes the event to an outbox table in the same transaction. A separate process reads the outbox and publishes events to the message broker. This avoids losing events if the broker is temporarily down.

Q200. What is blue green deployment?

Blue green deployment runs two identical environments: blue (current) and green (new). You deploy the new version to green, test it, and switch traffic from blue to green. If something breaks, you switch back instantly. It enables zero downtime deployments.

Q201. What is canary deployment?

Canary deployment rolls out a new version to a small percentage of users first. You monitor for errors and performance issues. If everything is healthy, you gradually increase traffic to the new version. If problems appear, you roll back only the canary.

Chapter 9: Distributed Systems and Consistency

Q202. What is the CAP theorem?

The CAP theorem states that a distributed system can guarantee only two of three properties at the same time: Consistency (all nodes see the same data), Availability (every request gets a response), and Partition Tolerance (system works despite network failures). Since partitions are unavoidable, the real choice is between CP and AP.

Q203. What is a CP system?

A CP system prioritizes consistency and partition tolerance. During a network partition, it may refuse to respond rather than serve stale data. Example: banking systems where showing an incorrect balance is worse than showing an error.

Q204. What is an AP system?

An AP system prioritizes availability and partition tolerance. During a network partition, it continues serving requests but some responses may have stale data. Example: social media feeds where showing a slightly old post is better than showing nothing.

Q205. What is the PACELC theorem?

PACELC extends CAP: if there is a Partition, choose between Availability and Consistency. Else (normal operation), choose between Latency and Consistency. Even without partitions, there is a tradeoff between how fast and how consistent the system is.

Q206. What is consensus in distributed systems?

Consensus is the process by which multiple nodes agree on a single value or decision. It is essential for leader election, transaction commits, and configuration changes. Achieving consensus in the presence of failures is one of the hardest problems in distributed computing.

Q207. What is the Raft consensus algorithm?

Raft is a consensus algorithm where nodes elect a leader through voting. The leader receives all writes and replicates them to followers. If the leader fails, a new election happens. Raft prioritizes understandability and is widely used in etcd, Consul, and CockroachDB.

Q208. What is the Paxos algorithm?

Paxos is the original consensus algorithm for distributed systems. It is mathematically proven to be correct but notoriously difficult to understand and implement. Most modern systems use Raft or ZAB instead because they are easier to work with.

Q209. What is leader election?

Leader election is the process by which distributed nodes choose one node to act as the coordinator. The leader handles writes or coordinates tasks. If the leader fails, a new election triggers automatically. Kafka, etcd, and ZooKeeper all use leader election.

Q210. What is a split brain problem?

Split brain occurs when a network partition causes two groups of nodes to each believe they are the leader. Both groups independently accept writes, leading to conflicting data. Quorum based systems prevent this by requiring a majority to agree before proceeding.

Q211. What is a quorum?

A quorum is the minimum number of nodes that must agree for an operation to succeed. In a cluster of N nodes, a quorum is typically (N/2) + 1. This ensures that any two quorums overlap by at least one node, preventing conflicting decisions.

Q212. What is vector clock?

A vector clock is a data structure that tracks causality between events in a distributed system. Each node maintains a counter, and the vector of all counters is attached to each message. It helps determine if events are causally related or concurrent.

Q213. What is consistent hashing?

Consistent hashing maps both servers and data keys onto a ring. Each key is assigned to the nearest server clockwise on the ring. When a server is added or removed, only a small fraction of keys need to be remapped. Used by DynamoDB, Cassandra, and CDNs.

Q214. What are virtual nodes in consistent hashing?

Virtual nodes assign each physical server multiple positions on the hash ring. This creates more even data distribution because a single server is responsible for many small segments rather than one large segment.

Q215. What is a gossip protocol?

A gossip protocol is a peer to peer communication method where each node periodically shares state information with a random subset of other nodes. Information spreads exponentially, like gossip in a social group. Used by Cassandra and DynamoDB for failure detection.

Q216. What is data partitioning?

Data partitioning divides data across multiple nodes so each node handles a subset. It improves performance by distributing the workload. The partition key determines which node stores each piece of data. The challenge is handling queries that span multiple partitions.

Q217. What is the two phase commit protocol (2PC)?

2PC is a protocol for achieving atomic transactions across multiple databases. Phase 1: the coordinator asks all participants to prepare. Phase 2: if all agree, the coordinator tells them to commit; if any disagree, all abort. It is slow and can block if the coordinator fails.

Q218. What is the three phase commit protocol (3PC)?

3PC adds a pre commit phase between prepare and commit to reduce the blocking problem in 2PC. If the coordinator fails after pre commit, participants can still make progress. However, 3PC is more complex and still cannot handle all failure scenarios.

Q219. What is eventual consistency?

Eventual consistency guarantees that if no new writes occur, all replicas will eventually converge to the same value. The time window of inconsistency is usually short (milliseconds to seconds). Suitable for systems where perfect freshness is not critical.

Q220. What is strong consistency?

Strong consistency guarantees that a read always returns the result of the most recent write. This requires coordination between nodes, which adds latency. It is essential for financial transactions and inventory management where stale reads cause real problems.

Q221. What is causal consistency?

Causal consistency ensures that causally related operations are seen in the correct order by all nodes. If user A posts a message and user B replies, all nodes see the post before the reply. Unrelated operations can appear in any order.

Q222. What is read your own writes consistency?

This guarantees that after a user makes a write, that same user always sees their own write in subsequent reads. Other users may still see stale data. Achieved by routing the user's reads to the same replica that handled their write.

Q223. What is linearizability?

Linearizability is the strongest consistency model. Every operation appears to happen atomically at some point between its start and end. It makes a distributed system behave as if there is only one copy of the data. It is expensive to achieve at scale.

Q224. What is the Byzantine Generals Problem?

It is a thought experiment about reaching consensus when some participants may be malicious or faulty and send conflicting information. Blockchain systems like Bitcoin solve this using proof of work. Traditional systems assume nodes are honest but may fail.

Q225. What is a distributed lock?

A distributed lock ensures that only one process across multiple servers can access a shared resource at a time. Redis (Redlock), ZooKeeper, and etcd are commonly used. The challenge is handling lock expiry, network delays, and ensuring safety during failures.

Q226. What is clock synchronization in distributed systems?

Different servers have slightly different clocks, which makes ordering events across machines difficult. NTP (Network Time Protocol) syncs clocks but is not perfectly accurate. Logical clocks (Lamport timestamps, vector clocks) provide ordering without relying on physical time.

Q227. What is the Lamport timestamp?

A Lamport timestamp is a logical clock that assigns a monotonically increasing counter to events. When a message is sent, the sender's counter is attached. The receiver updates its counter to be greater than both its own and the received value. It establishes a partial order of events.

Q228. What is a conflict free replicated data type (CRDT)?

A CRDT is a data structure that can be replicated across multiple nodes and updated independently without coordination. Updates are merged automatically and always converge to the same state. Used in collaborative editing tools and distributed counters.

Q229. What is data replication and why is it needed?

Data replication copies data to multiple servers for fault tolerance (if one server dies, others have the data), read performance (distribute read load), and geographic locality (serve data from servers close to users). The tradeoff is keeping replicas in sync.

Q230. What is multi region deployment?

Multi region deployment runs your application and data across multiple geographic regions. It reduces latency for global users and provides disaster recovery. The challenge is keeping data consistent across regions with network latency between them.

Q231. What is a write quorum and read quorum?

In a system with N replicas, a write quorum (W) is the minimum number of replicas that must acknowledge a write. A read quorum (R) is the minimum number of replicas that must respond to a read. If W + R > N, you get strong consistency.

Chapter 10: Security and Authentication

Q232. What is authentication vs authorization?

Authentication verifies who you are (login with username and password). Authorization determines what you are allowed to do (can this user delete this resource?). Authentication comes first. Authorization comes after.

Q233. What is OAuth 2.0?

OAuth 2.0 is an authorization framework that lets a third party application access a user's data without exposing their password. When you click 'Login with Google', the app gets a token from Google that grants limited access to your account.

Q234. What is JWT?

JWT (JSON Web Token) is a compact, self contained token used for authentication and information exchange. It contains encoded user claims (like user ID and role), is signed to prevent tampering, and is stateless so the server does not need to store session data.

Q235. What is the difference between cookies and tokens?

Cookies are stored in the browser and sent automatically with every request to the same domain. Tokens (like JWTs) are stored by the application and sent explicitly in the Authorization header. Tokens are more flexible for mobile apps and cross domain requests.

Q236. What is session based authentication?

The server creates a session after login and stores it in memory or a database. The client receives a session ID stored in a cookie. On each request, the server looks up the session. This requires server side storage and does not scale well across multiple servers without shared storage.

Q237. What is token based authentication?

After login, the server issues a signed token (like JWT) containing user information. The client sends this token with every request. The server validates the token without looking anything up. This is stateless and scales easily across multiple servers.

Q238. What is SSO (Single Sign On)?

SSO allows a user to log in once and access multiple applications without logging in again. When you log into Gmail and can access YouTube and Google Drive without re authenticating, that is SSO. It is typically implemented using OAuth 2.0 or SAML.

Q239. What is two factor authentication?

Two factor authentication (2FA) requires two forms of identity verification: something you know (password) and something you have (phone, hardware key) or something you are (fingerprint). It significantly reduces the risk of unauthorized access.

Q240. What is API key authentication?

An API key is a unique identifier passed with each request to identify the calling application. It is simple but not very secure since keys can be leaked. Use it for server to server communication or public APIs with rate limiting. Do not use it as the sole authentication for sensitive operations.

Q241. What is HTTPS and why is it important?

HTTPS encrypts all data between the client and server using TLS. Without it, passwords, credit cards, and personal data travel as plain text across the internet. Anyone on the same network could intercept and read it.

Q242. What is a DDoS attack?

A DDoS (Distributed Denial of Service) attack overwhelms a server with massive traffic from many sources, making it unavailable to legitimate users. Defenses include rate limiting, CDNs that absorb traffic, IP blacklisting, and services like Cloudflare and AWS Shield.

Q243. What is SQL injection?

SQL injection is an attack where malicious SQL code is inserted into input fields to manipulate the database. For example, entering ' OR 1=1 in a login form to bypass authentication. Prevent it by using parameterized queries and never concatenating user input into SQL statements.

Q244. What is encryption at rest and in transit?

Encryption in transit protects data as it moves over the network (HTTPS/TLS). Encryption at rest protects data stored on disk (database encryption, file system encryption). Both are needed for comprehensive data security.

Q245. What is the principle of least privilege?

Every user, service, or process should have only the minimum permissions needed to perform its task. A notification service should not have write access to the payment database. This limits the damage if any component is compromised.

Q246. What is mutual TLS (mTLS)?

In regular TLS, only the server proves its identity to the client. In mTLS, both the client and server authenticate each other using certificates. It is commonly used in microservices environments (via service meshes like Istio) to secure service to service communication.

Chapter 11: Monitoring, Logging, and Observability

Q247. What is observability?

Observability is the ability to understand the internal state of a system by examining its external outputs. The three pillars are logs (what happened), metrics (how much or how fast), and traces (the path a request took across services).

Q248. What is the difference between monitoring and observability?

Monitoring tells you when something is wrong by tracking predefined metrics and triggering alerts. Observability helps you understand why something is wrong by giving you the tools to explore and diagnose issues you did not anticipate.

Q249. What are the three pillars of observability?

Logs: detailed records of discrete events (error messages, request details). Metrics: numerical measurements over time (CPU usage, request latency, error rates). Traces: the journey of a single request across multiple services with timing data.

Q250. What is distributed tracing?

Distributed tracing follows a request as it travels through multiple services. A unique trace ID is passed along with the request. Each service logs a span (its processing time and outcome). The spans are stitched together to visualize the full request path and identify bottlenecks.

Q251. What is a health check endpoint?

A health check endpoint (like /health or /status) returns the health status of a service. A basic check confirms the service is running. A deep check verifies that the service can reach its database, cache, and other dependencies.

Q252. What is a heartbeat mechanism?

A heartbeat is a periodic signal sent by a server to a monitoring system to indicate it is alive. If the monitoring system misses several consecutive heartbeats, it marks the server as unhealthy and triggers failover or alerts.

Q253. What are SLIs, SLOs, and SLAs?

SLI (Service Level Indicator) is a measured metric like request latency. SLO (Service Level Objective) is the target for that metric like p99 latency under 200ms. SLA (Service Level Agreement) is a formal contract with penalties if the SLO is not met.

Q254. What is log aggregation?

Log aggregation collects logs from all services and servers into a centralized system for searching and analysis. Without it, debugging requires SSH into individual servers. Tools: ELK Stack (Elasticsearch, Logstash, Kibana), Datadog, Splunk, Grafana Loki.

Q255. What is alerting and what makes a good alert?

Alerting notifies engineers when metrics cross predefined thresholds. Good alerts are actionable (someone can do something about it), urgent (it affects users now or will soon), and not noisy (too many false alarms cause alert fatigue and important alerts get ignored).

Q256. What is the difference between p50, p95, and p99 latency?

p50 is the median latency (50% of requests are faster). p95 means 95% of requests are faster. p99 means 99% of requests are faster. p99 catches the tail latency that affects a small but important percentage of users. Always monitor p99 for user facing services.

Q257. What is Prometheus?

Prometheus is an open source metrics collection and alerting system. It scrapes metrics from services at regular intervals and stores them in a time series database. It uses PromQL for querying and integrates with Grafana for visualization and Alertmanager for alerts.

Q258. What is Grafana?

Grafana is an open source visualization and dashboarding tool. It connects to data sources like Prometheus, Elasticsearch, and CloudWatch to create real time dashboards showing metrics, logs, and traces in a unified view.

Q259. What is OpenTelemetry?

OpenTelemetry is a vendor neutral, open source standard for collecting telemetry data (traces, metrics, logs). It provides libraries and agents that instrument your application and send data to any compatible backend like Jaeger, Zipkin, or Datadog.

Q260. What is error budgeting?

An error budget is the allowed amount of unreliability based on your SLO. If your SLO is 99.9% uptime, your error budget is 0.1% (about 8.7 hours per year). If you exhaust your budget, you prioritize reliability work over new features until the budget recovers.

Q261. Why is monitoring critical for distributed systems?

In a monolith, problems are localized. In distributed systems, a latency spike in one service can cascade across dozens of others. Without monitoring, you cannot identify which service, database query, or network hop is causing the problem.

Chapter 12: Design Patterns and Best Practices

Q262. What is the fan out pattern?

Fan out distributes a single task to multiple workers or services in parallel. When a user posts on social media, the post needs to be added to the feeds of all their followers. Fan out on write pushes to all followers immediately. Fan out on read assembles the feed when the follower opens the app.

Q263. What is fan out on write vs fan out on read?

Fan out on write precomputes and pushes data to all recipients when an event occurs (fast reads, expensive writes). Fan out on read computes data on demand when a recipient requests it (cheap writes, expensive reads). Instagram uses a hybrid: fan out on write for most users, fan out on read for celebrity accounts.

Q264. What is the CQRS pattern?

CQRS separates the system into a command side (handles writes) and a query side (handles reads). Each side can use different databases, different data models, and scale independently. Events synchronize the two sides. It is ideal for read heavy systems.

Q265. What is event sourcing?

Instead of storing only the current state, event sourcing stores every state change as an immutable event. The current state is derived by replaying events. It provides a complete audit trail and enables time travel (reconstructing state at any past point).

Q266. What is the singleton pattern in distributed systems?

In distributed systems, ensuring only one instance of a process runs across the cluster requires distributed coordination. Use leader election (via ZooKeeper or etcd) to designate one node as the singleton. If that node fails, another takes over.

Q267. What is the read replica pattern?

Route read queries to replica databases and write queries to the primary. This scales read capacity linearly by adding replicas. The tradeoff is replication lag: reads from replicas may return slightly stale data.

Q268. What is the write behind pattern?

The application writes to a fast store (like a cache) and the data is asynchronously persisted to the primary database later. This makes writes extremely fast but risks data loss if the fast store crashes before the data is persisted.

Q269. What is the materialized view pattern?

Precompute and store the result of complex queries in a separate table or service. When the underlying data changes, the materialized view is updated. This makes reads very fast for expensive aggregations or joins that would be slow to compute on every request.

Q270. What is the throttling pattern?

Throttling limits the rate at which requests are processed to protect the system. Unlike rejection (returning errors), throttling can queue requests, slow down processing, or return degraded responses. It smooths traffic spikes and prevents cascading failures.

Q271. What is the ambassador pattern?

An ambassador is a helper process that handles outgoing network communication on behalf of the main application. It manages retries, timeouts, circuit breaking, and logging. This offloads resilience logic from the application and keeps the codebase clean.

Q272. What is the anti corruption layer pattern?

An anti corruption layer is an adapter between your new system and a legacy system. It translates requests and responses between the two, preventing the legacy system's design decisions from corrupting your new system's architecture.

Q273. What is the competing consumers pattern?

Multiple consumers compete to process messages from a single queue. Each message is processed by exactly one consumer. Adding more consumers increases throughput. If one consumer fails, others continue processing. This is the standard pattern for task distribution.

Q274. What is the priority queue pattern?

Messages are assigned different priorities, and higher priority messages are processed before lower priority ones. This ensures critical operations (payment processing) are handled before non critical ones (sending analytics events) during high load.

Q275. What is data locality?

Data locality means processing data on or near the server where it is stored, rather than moving large amounts of data across the network. It reduces latency and bandwidth usage. MapReduce and Spark move computation to the data nodes for this reason.

Q276. What is the claim check pattern?

Instead of sending a large message through the queue, store the payload in a blob store and send only a reference (the claim check) through the queue. The consumer uses the reference to retrieve the full payload. This keeps queues fast and lightweight.

Q277. What is sharding and when should you use it?

Sharding splits a database across multiple servers based on a key. Use it when a single server cannot handle the data volume or query load, and vertical scaling has reached its limit. Avoid it when the added complexity of cross shard queries and rebalancing is not justified.

Q278. What is the difference between horizontal and vertical partitioning?

Horizontal partitioning (sharding) splits rows across servers. All columns stay together. Vertical partitioning splits columns across tables or servers. Frequently accessed columns are grouped together for faster queries.

Q279. What is the batch processing pattern?

Collect data over a period and process it all at once. Used for nightly reports, monthly billing, and data warehouse ETL jobs. Tools: Hadoop, Spark. It is efficient for large volumes but introduces delay between data arrival and processing.

Q280. What is the stream processing pattern?

Process data continuously as it arrives in real time. Used for fraud detection, live dashboards, and real time recommendations. Tools: Kafka Streams, Flink, Spark Streaming. It provides immediate results but is more complex to build and maintain.

Q281. What is the Lambda architecture?

Lambda architecture combines batch and stream processing. A speed layer provides real time approximate results. A batch layer reprocesses the same data for accurate results. A serving layer merges both. It provides both speed and accuracy but maintains two separate pipelines.

Chapter 13: Real World System Design Problems

Q282. How would you design a URL shortener like bit.ly?

Use a hash or counter based approach to generate short unique IDs. Store the mapping (short URL to long URL) in a database with the short URL as the primary key. Use a cache (Redis) for frequently accessed URLs. A load balancer distributes traffic, and a 301 redirect sends users to the original URL.

Q283. How would you design a rate limiter?

Use the token bucket or sliding window algorithm. Store counters per user or IP in Redis for fast lookups. Place the rate limiter at the API Gateway so all requests pass through it. Return HTTP 429 when the limit is exceeded and include a Retry-After header.

Q284. How would you design a chat system like WhatsApp?

Use WebSockets for real time bidirectional communication. A chat service routes messages between users. Store messages in a NoSQL database (Cassandra) for high write throughput. Use message queues for reliability. For offline users, store messages and deliver them when they reconnect.

Q285. How would you design a notification system?

Accept notification requests through an API and push them to a message queue. Different consumers handle different channels (push, email, SMS). Use templates for formatting. Track delivery status in a database. Implement retry logic for failed deliveries and rate limit to avoid spamming users.

Q286. How would you design a news feed system like Instagram?

Use a hybrid fan out approach. For regular users, precompute the feed on write by pushing new posts to followers' feed caches. For celebrity accounts with millions of followers, compute the feed on read to avoid expensive writes. Store feeds in Redis for fast access.

Q287. How would you design a video streaming service like YouTube?

Upload videos to blob storage. Encode them into multiple resolutions and formats using a processing pipeline. Distribute encoded files across CDN edge servers. Use adaptive bitrate streaming so the player switches quality based on network speed. Store metadata in a database and use a search index for discovery.

Q288. How would you design a search autocomplete system?

Build a trie (prefix tree) data structure containing popular search terms weighted by frequency. When the user types a prefix, traverse the trie to find top matching suggestions. Cache popular prefixes in memory. Update the trie periodically using analytics data. Distribute the trie across servers for scale.

Q289. How would you design a distributed cache?

Use consistent hashing to distribute keys across cache servers. When a server is added or removed, only a small portion of keys need to be remapped. Use replication for fault tolerance. Implement eviction policies (LRU) when memory is full. Monitor cache hit rates and add servers as needed.

Q290. How would you design a file storage system like Google Drive?

Store files in a distributed object store (like S3). Use chunking to split large files into smaller pieces for efficient upload, download, and deduplication. Track metadata (file name, owner, permissions) in a relational database. Use WebSockets or long polling to sync changes across devices in real time.

Q291. How would you design an e-commerce system like Amazon?

Decompose into microservices: product catalog, search, cart, order, payment, inventory, shipping, and notifications. Use a relational database for orders and payments. Use Elasticsearch for product search. Use message queues to decouple order processing. Use caching heavily for the product catalog.

Q292. How would you design a ride sharing service like Uber?

Track driver locations using real time GPS updates sent every few seconds. Use geospatial indexing (S2 geometry or H3) to find nearby drivers. A matching algorithm considers distance, traffic, and driver direction. Use message queues for reliability. Calculate ETA using ML models trained on historical data.

Q293. How would you design a web crawler?

Start with seed URLs. A URL frontier (priority queue) manages which URLs to crawl next. Worker nodes fetch pages, extract links, and add new URLs to the frontier. Use a bloom filter to avoid revisiting URLs. Store crawled content for indexing. Respect robots.txt and implement politeness policies.

Q294. How would you design a payment system?

Use a state machine to track payment states (initiated, authorized, captured, settled, refunded, failed). Ensure idempotency using unique payment IDs. Log every state transition with a write ahead log. Use retries with backoff for external API calls. Reconcile records with bank statements daily.

Q295. How would you design a social media platform like Twitter?

Users service handles profiles and follows. Tweet service stores tweets with timestamps. Timeline service assembles feeds using fan out. Use a cache layer for hot timelines. Store tweets in a distributed database partitioned by user ID. Use Kafka for event distribution and Elasticsearch for search.

Q296. How would you design a hotel booking system like Airbnb?

A listing service stores property details with geospatial data. A search service indexes listings by location, price, and dates. A booking service handles reservations using pessimistic locking to prevent double bookings. A payment service handles transactions. Use message queues for confirmation emails and host notifications.

Q297. How would you design a content delivery network (CDN)?

Deploy edge servers in multiple geographic locations. When content is requested, route the user to the nearest edge server via DNS. If the edge has the content cached, serve it directly. If not, fetch from the origin, cache it, and serve. Use TTLs to manage freshness and cache invalidation for updates.

Q298. How would you design a parking lot system?

Track available spots in real time using a database. Each parking level and spot has a status (available, occupied, reserved). Assign spots based on vehicle type and proximity to entrance. Use sensors or ticket systems at entry/exit. Display availability on signs. Calculate fees based on duration.

Q299. How would you design an online multiplayer game?

Use UDP for real time game state updates (speed over reliability). A game server maintains the authoritative state. Use client side prediction to hide latency. A matchmaking service groups players by skill and region. Use WebSockets for lobby chat. Store player profiles and stats in a database.

Q300. How would you design a distributed logging system?

Application servers write logs to local buffers. A log shipping agent (Fluentd, Filebeat) sends logs to a centralized broker (Kafka). Log processing pipelines parse, filter, and enrich logs before storing them in a searchable store (Elasticsearch). Grafana or Kibana provides the dashboard for querying and visualization.

Q301. How would you design a key value store like DynamoDB?

Use consistent hashing to distribute keys across nodes. Replicate data across multiple nodes for durability. Use vector clocks to handle conflicting writes. Implement read and write quorums for tunable consistency. Use a gossip protocol for cluster membership and failure detection. Support range queries with sorted storage.

Q302. How would you design Google Maps?

Store road network data as a graph. Use graph algorithms (Dijkstra or A*) for routing. Collect real time GPS data from users to compute live traffic. Partition map data by geographic tiles for efficient rendering. Use a CDN to serve map tiles. ML models predict traffic and estimate ETAs.

Q303. How would you design a collaborative editing tool like Google Docs?

Use Operational Transformation (OT) or CRDTs to merge concurrent edits from multiple users without conflicts. Each user has a local copy. Edits are sent to the server and broadcast to other users via WebSockets. Store document versions for undo/redo and history. Use a permission system for sharing.

Q304. How would you design a ticket booking system like BookMyShow?

Show available events with seat maps. When a user selects seats, temporarily lock them for a few minutes. Use pessimistic locking in the database to prevent double booking. Process payment within the lock window. If payment fails or times out, release the seats. Use queues during high demand to manage traffic.

Q305. How would you design a food delivery system like Zomato?

Restaurant service manages menus and availability. Search service indexes restaurants by location and cuisine. Order service manages the order lifecycle. A matching service assigns the nearest available delivery partner using geospatial queries. Real time tracking uses WebSockets. Payment and notification services run independently.

Q306. How would you design a metrics collection and alerting system?

Applications expose metrics via endpoints or push them to a collector. A time series database (InfluxDB, Prometheus) stores the data. An alerting engine evaluates rules against metrics and

triggers notifications via email, Slack, or PagerDuty. Grafana dashboards visualize trends and anomalies.